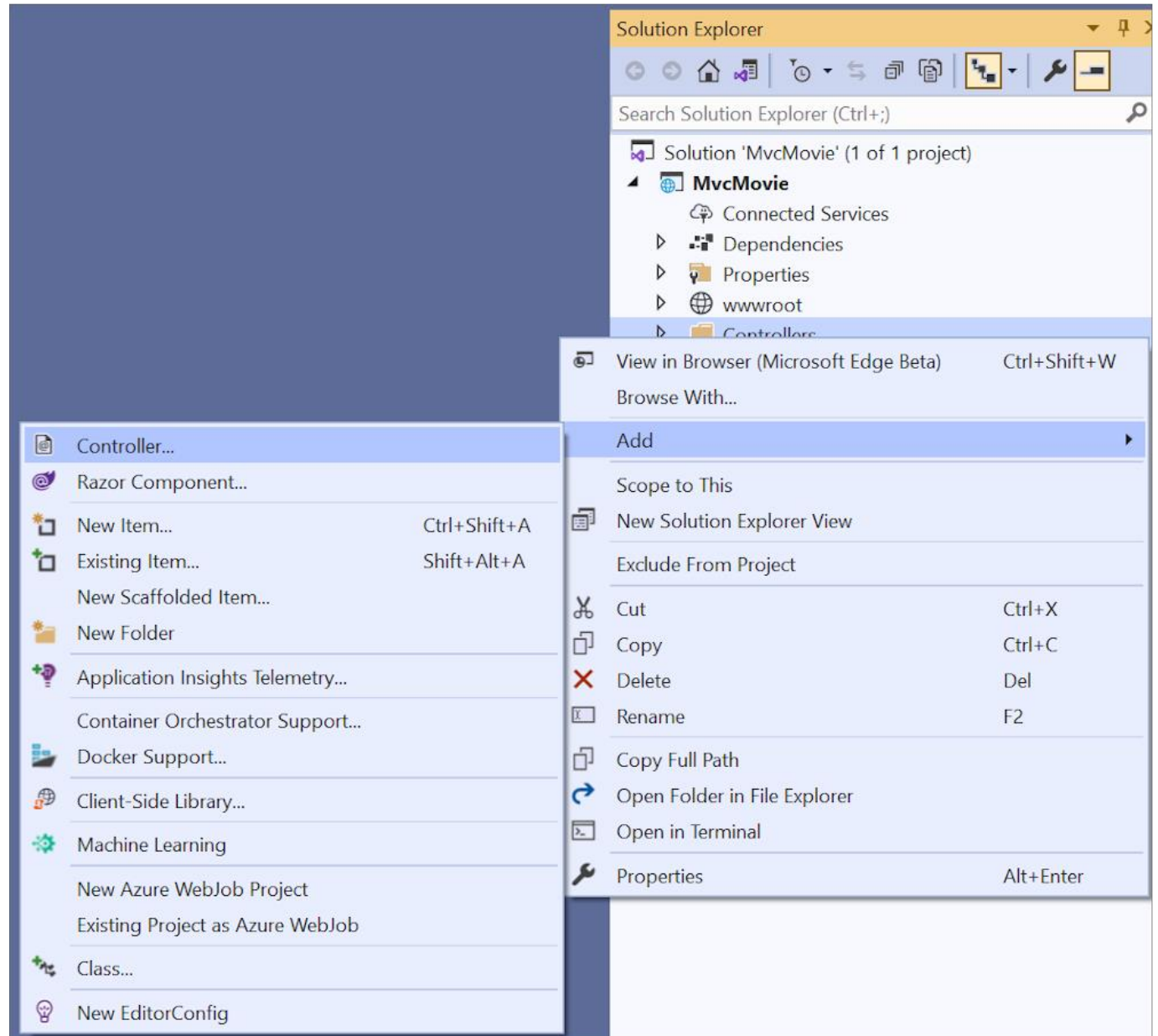


Controller

- ◆ The Model-View-Controller (MVC) architectural pattern separates an app into three main components: Model, View, and Controller. The MVC pattern helps you create apps that are more testable and easier to update than traditional monolithic apps.
- ◆ Models: Classes that represent the data of the app. The model classes use validation logic to enforce business rules for that data. Typically, model objects retrieve and store model state in a database. In this tutorial, a Movie model retrieves movie data from a database, provides it to the view or updates it. Updated data is written to a database.
- ◆ Views: Views are the components that display the app's user interface (UI). Generally, this UI displays the model data.
- ◆ Controllers: Classes that:
 - ◆ Handle browser requests.
 - ◆ Retrieve model data.
 - ◆ Call view templates that return a response.

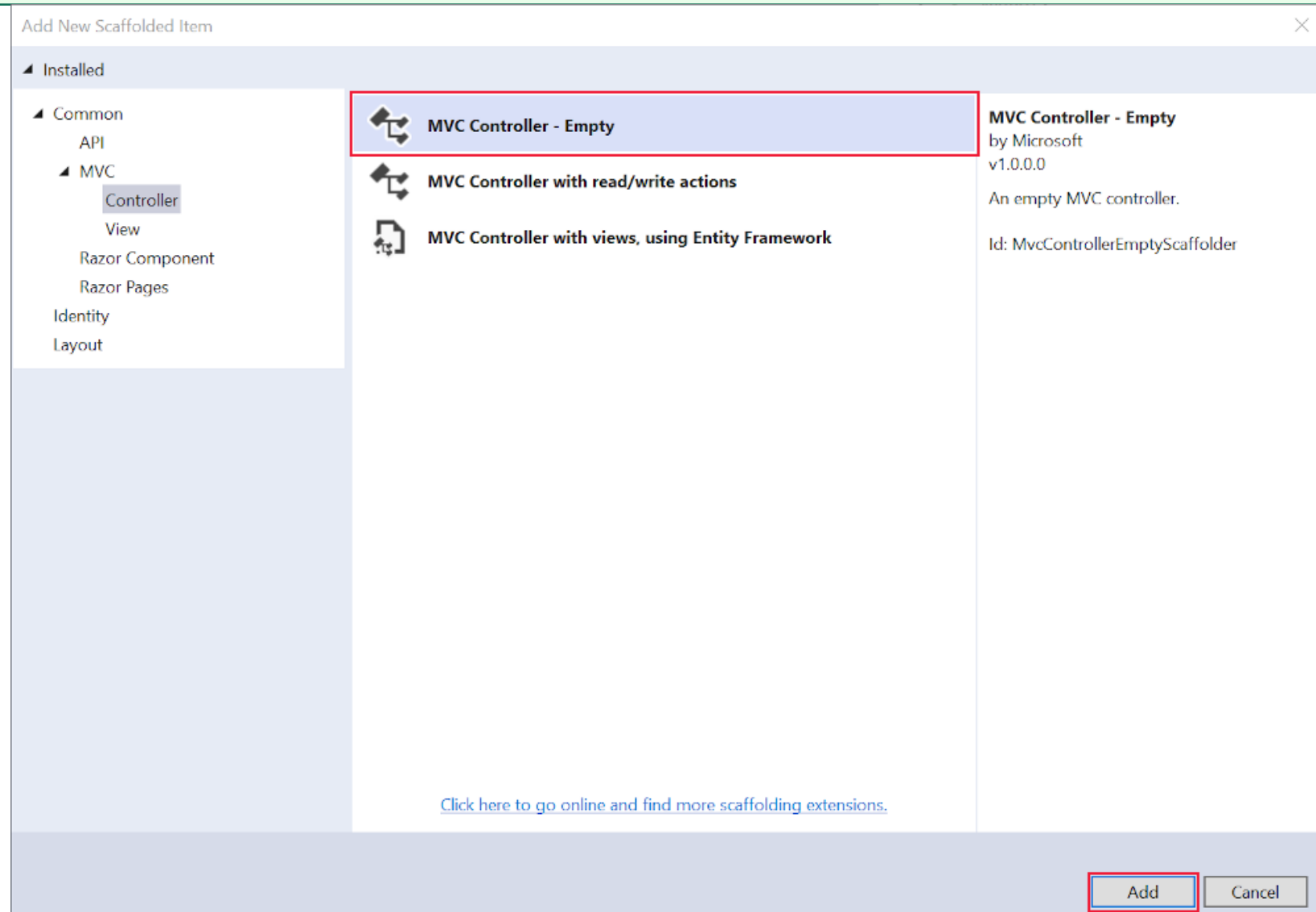
Add a controller

right-click Controllers >
Add > Controller



Add a controller

In the Add New Scaffolded Item dialog box, select MVC Controller - Empty>Add.



Add a controller

- ◆ Enter HelloWorldController.cs and select Add.
- ◆ An HTTP endpoint: Is a targetable URL in the web application, such as: `https://localhost:5001/HelloWorld`
- ◆ Combines:
 - ◆ The protocol used: HTTPS.
 - ◆ The network location of the web server, including the TCP port: `localhost:5001`.
 - ◆ The target URI: `HelloWorld`

```
using Microsoft.AspNetCore.Mvc;
using System.Text.Encodings.Web;

namespace MvcMovie.Controllers
{
    public class HelloWorldController : Controller
    {
        //
        // GET: /HelloWorld/

        public string Index()
        {
            return "This is my default action...";
        }

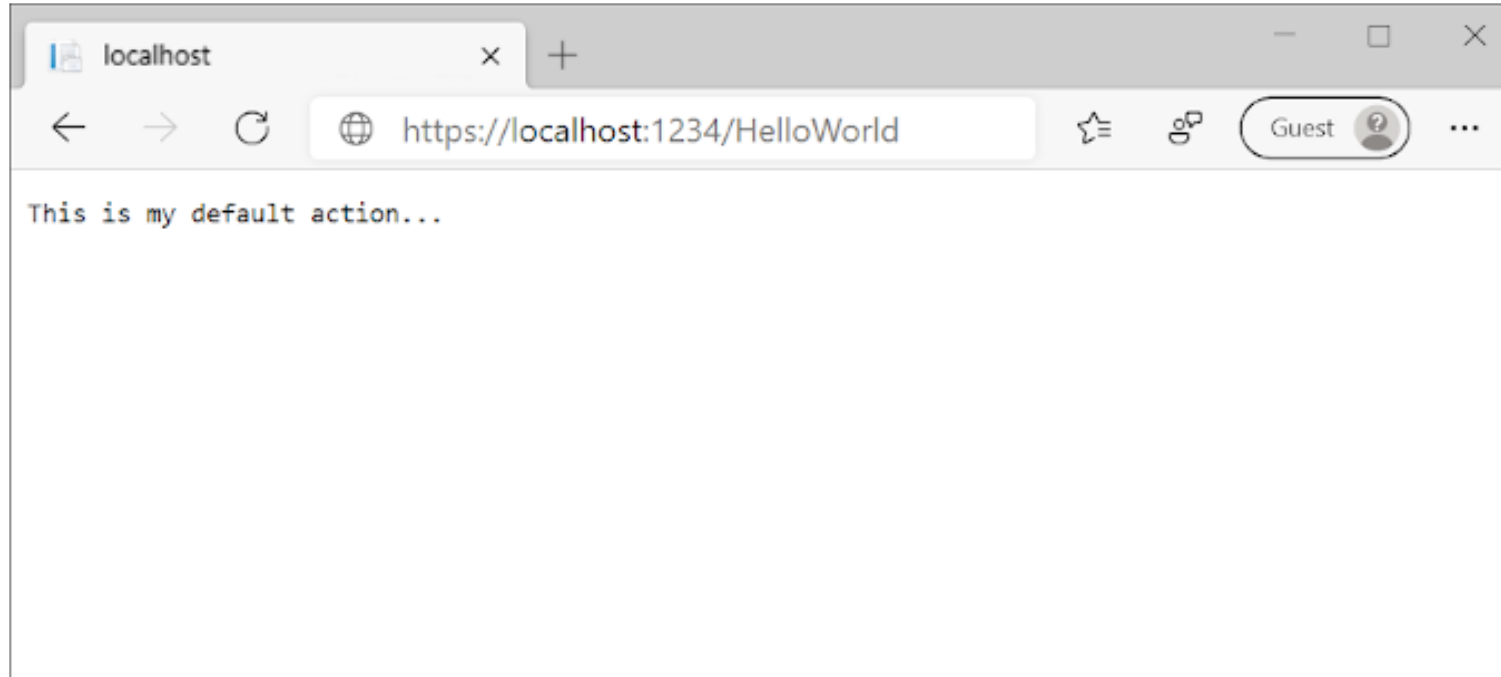
        //
        // GET: /HelloWorld/Welcome/

        public string Welcome()
        {
            return "This is the Welcome action method...";
        }
    }
}
```

Add a controller

- ◆ The first comment states this is an HTTP GET method that's invoked by appending /HelloWorld/ to the base URL.
- ◆ The second comment specifies an HTTP GET method that's invoked by appending /HelloWorld/Welcome/ to the URL. Later on in the tutorial, the scaffolding engine is used to generate HTTP POST methods, which update data.

Add a controller



- ◆ MVC invokes controller classes, and the action methods within them, depending on the incoming URL. The default URL routing logic used by MVC, uses a format like this to determine what code to invoke:

`/[Controller]/[ActionName]/[Parameters]`

- ◆ The routing format is set in the Program.cs file.

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Pass parameter from the URL to the controller

- ◆ Change the Welcome method to include two parameters as shown in the following code.

```
// GET: /HelloWorld/Welcome/  
// Requires using System.Text.Encodings.Web;  
public string Welcome(string name, int numTimes = 1)  
{  
    return HtmlEncoder.Default.Encode($"Hello {name}, NumTimes is:  
{numTimes}");  
}
```

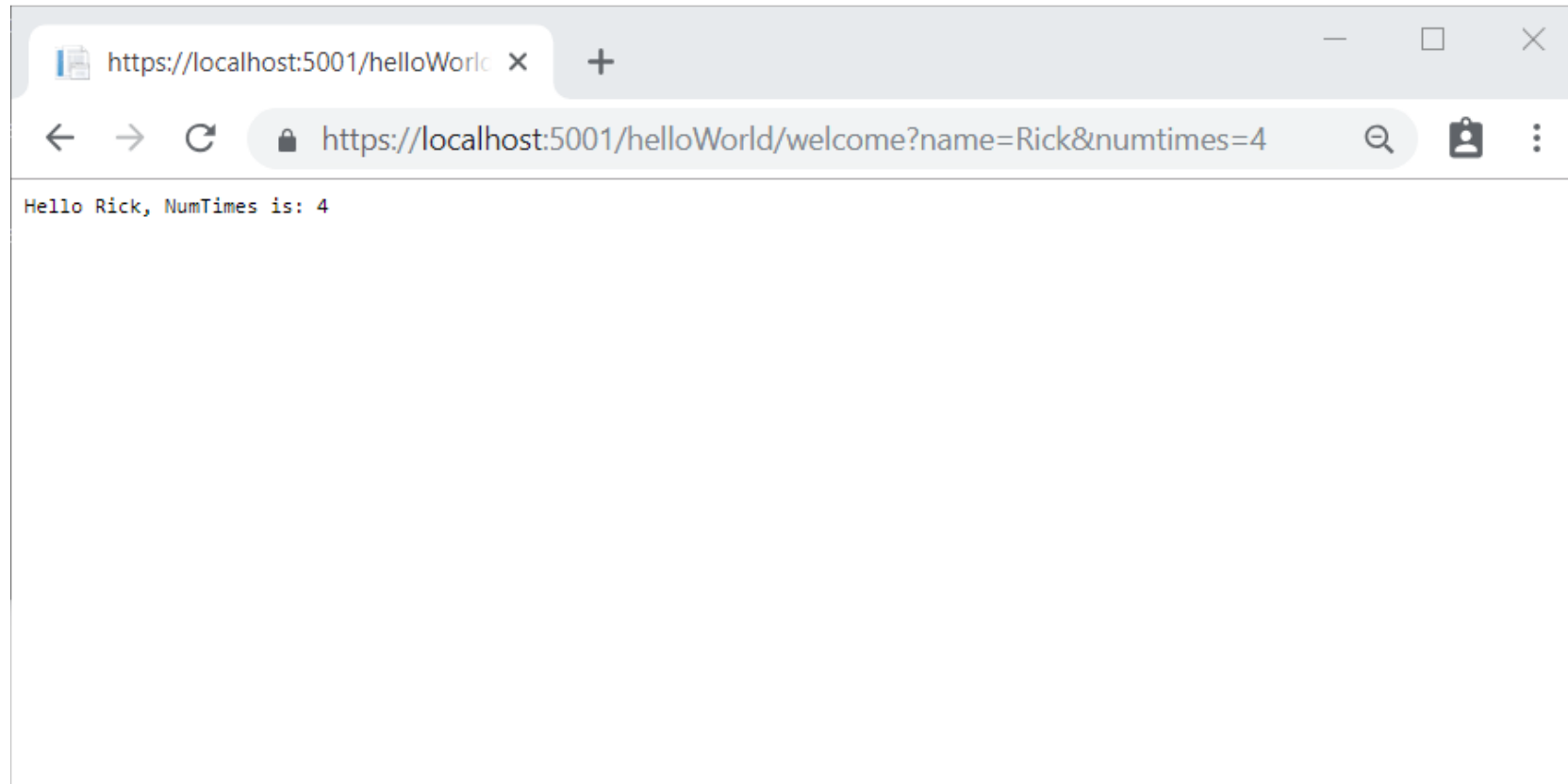
- ◆ Uses the C# optional-parameter feature to indicate that the numTimes parameter defaults to 1 if no value is passed for that parameter.
- ◆ Uses Html Encoder.Default.Encode to protect the app from malicious input, such as through JavaScript.
- ◆ Uses Interpolated Strings in \$"Hello {name}, NumTimes is: {numTimes}"

Pass parameter from the URL to the controller

◆ Run the app and browse to

`https://localhost:{PORT}/HelloWorld/Welcome?name=Rick&numtimes=4`

◆ Replace {PORT} with your port number.



Pass parameter from the URL to the controller

- ◆ Replace the Welcome method with the following code:

```
public string Welcome(string name, int ID = 1)
{
    return HtmlEncoder.Default.Encode($"Hello {name}, ID: {ID}");
}
```

- ◆ Run the app and enter the following URL:

<https://localhost: {PORT}/HelloWorld/Welcome/3?name=Rick>

Pass parameter from the URL to the controller

- ◆ In the preceding URL:
- ◆ The third URL segment matched the route parameter id.
- ◆ The Welcome method contains a parameter id that matched the URL template in the MapControllerRoute method.
- ◆ The trailing ? starts the query string.